

ActivateU

Authors

Matt ter Steege

m.j.tersteege@students.uu.nl

Max Harmsen

m.harmsen1@uu.nl

Contents

1	User stories	2
1.1	Lidija Dudek — The beginner	2
1.2	Marinos Lamberti — The intermediate	3
1.3	Alexandria Kozlova — The expert	3
2	Non-functional requirements	4
3	Clarifications	4
4	Domain model	5
4.1	Lidija Dudek	5
4.2	Alexandria Kozlova	6
5	Activity diagram	7
5.1	Track Activity	7
5.2	Friends and sharing	8
6	Software design	9
7	Evaluation	10
7.1	Variation	10
7.2	Future changes	10
7.3	Notification center	11
7.4	Cohesion en coupling	11
8	Work distribution & reflection	11
8.1	Work distribution	11
8.2	Reflection	12
A	Mermaid diagrams	13
A.1	Lidija Dudek	13
A.2	Alexandria Kozlova	14
A.3	Class diagram	15

Introduction

This project focuses on the design of a minimal WorkoutApp: **ActivateU**. The app is not only intended to help people who don't like sports become more active in a simple and motivating way, but also for people who sport more often but don't need a bunch of unnecessary extras around their activities. The main focus of the project is tracking running and walking activities, allowing users to easily record their time, distance, and pace. The app should encourage users through realistic goals, progress tracking, and positive feedback, without overwhelming them with complex fitness features. In addition, the application should be designed with future integrations, such as wearable devices and community features, in mind.

1 | User stories

The user stories below are defined in an accumulative manner, so the beginner scenarios are still relevant for the intermediate and expert users.

Lidija Dudek — The beginner

Lidija lives a semi-healthy lifestyle, but does sometimes reach for fast food. She wants to sport more often – with running and walking being the best option. She says “Just putting on shoes and going outside” feels like a big step, and she is looking to close that gap and fit sporting more into her life.

As a beginner runner, I want to start tracking my walk or run with one tap, so that I can start exercising without complicated setup.

Scenario (a): The user clicks the start walk/run button

Outcome (a): The application shows the activity view and the application starts recording location data

As a beginner runner, I want to be able to pause my activity, so that I can catch my breath.

Scenario (b): The user clicks the pause button on the activity view

Outcome (b): The application pauses the recording of the activity/location data and waits till the activity is resumed

As a beginner runner, I want to be able to stop my activity, so that I can stop when I feel I've done enough.

Scenario (c): The user clicks the finish button

Outcome (c): The application ends the activity/location tracking and transforms it into usable data

As a beginner runner, I want to be able to share my accomplishments with people, so that they can help me improve.

Scenario (d): The user clicks the share run/walk button

Outcome (d): The application shows the native share window to send the activity data via major social media platforms.

As an extrovert, I want to be able to find and add friends, so that I can see what they are achieving.

Scenario (e): The user types in a username and clicks add friend

Outcome (e): The application checks if the username is valid and proceeds to create a friend request on the other user's account

Marinos Lamberti — The intermediate

Marinos adopted a more active lifestyle around 2 years ago on advice of a doctor who had slight concerns about his weight. However, after some months he really got the hang of it. He enjoyed seeing his progress, and he actually did like running a lot. He downloaded a mobile application to keep track of his progress, but after a year of use, he noticed that he was overwhelmed by the constant updates of the application. He didn't want the ability to add swimming training, nor a forced AI trainer that critiques every move.

As an intermediate runner, I want to be able to start my run wherever and whenever I want, so that I am not limited by the application.

Scenario (f): The user wants to start a run at any location. This includes locations such as the park where there is possibly no internet connection.

Outcome (f): The application must allow the user to start, track, and save a run while offline.

As an intermediate runner, I want to be able to stop AI from analysing my every move, so that I can do it myself.

Scenario (g): The user turns off the AI trainer to focus on running.

Outcome (g): The application allows the user to disable the AI trainer without disabling other running functionality.

As an intermediate runner, I want some insights into my achievements, so that I can have a better understanding of my own achievements.

Scenario (h): The user wants to look up the distance he ran over the past week in the insights tab.

Outcome (h): The application shall provide a clear overview of the distance that is run in the past week, timings of those runs and the speed graph of each run.

Alexandria Kozlova — The expert

Alexandria is a hardcore runner; she runs every other day and does running races every 2 weeks. She does not start runs that are shorter than 30 kilometres. This habit of her is already 35 years old, meaning Alexandria herself is around 60 years old. Therefore, she didn't grow up with a mobile phone. She also has difficulties with the Dutch language, since she moved to the Netherlands 3 years ago.

As an expert runner, I want to know the exact route I've run, so that I can track what I have done in greater detail.

Scenario (i): The user wants to start a run in the forest with twisting, non-straight paths.

Outcome (i): The application tracks her location in great precision (<10m)

As someone who is computer-illiterate, I want the application to help me understand how to use it, so that I am not completely lost when using the application.

Scenario (j): The user opens the app for the first time.

Outcome (j): The application shows her where to find everything so she can easily use the application the next time.

As someone who does not speak Dutch very well, I want to be able to change the app language, so that I can understand the application better.

Scenario (k): The user goes into settings to change the language of the application to her native language.

Outcome (k): The application changes the text on the views to her native selected language.

As an expert runner, I want to be able to also record activities using my smartwatch, so that I can leave my phone at home.

Scenario (l): The user connects the application with her smartwatch and starts running without her phone.
Outcome (l): The smartwatch uses its automatic activity detection to track her run and synchronises it again when she gets home.

As someone who is computer-illiterate, I want the deletion of my account to be fast and clear, so that I can know for sure my data is nowhere it should not be.

Scenario (m): The user opens the application settings, scrolls down and clicks “Delete account” and confirms via a pop-up.
Outcome (m): The application sends an account deletion request to the server, which then marks the account for deletion and wipes it after 1 week (assuming no cancel event happened).

2 | Non-functional requirements

- NFR-1 **Performance** The application shall load any view within 3 seconds on high- to lower-end smartphones in 95% of the time.
- NFR-2 **Concurrent** The application shall support 1.000.000 registered users and 100.000 concurrent users while maintaining a response time of ≤ 2 seconds for 95% of requests under normal operating conditions.
- NFR-3 **Uptime** The application shall maintain at least 99.5% uptime under normal operating conditions.
- NFR-4 **Development** The application shall be developed using short, iterative development increments guarded by a product owner and scrum master.
- NFR-5 **MVP** The MVP shall only provide functionality for tracking a walking/running activity.
- NFR-6 **Protection** (Sensitive) personal and activity data shall be protected against unauthorised access using encrypted storage/transmission protocols like HTTPS and AES256.
- NFR-7 **GDPR** The application shall comply with applicable privacy legislation, including GDPR.
- NFR-8 **Architecture** The codebase shall be maintainable, modular and extendable so as not to disrupt normal application operations as described in ISO/IEC/IEEE 24765:2017.
- NFR-9 **Documentation** The application shall be documented to support future development and maintenance.
- NFR-10 **Battery** The application shall not unnecessarily synchronise data with the servers to keep battery usage below 2% battery drain per hour on batteries in normal condition.
- NFR-11 **Accessibility** The application shall at least meet the WCAG A standard and shall support the 5 most common languages in the world.
- NFR-12 **Maintainability** The frontend and backend shall use actively maintained frameworks or programming languages, where the last update happened less than one year ago.
- NFR-13 **Deletion** The application shall provide an easy and accessible way to remove an account with all its corresponding (personal) data within 1 week of the request.
- NFR-14 **Connectivity** The application shall have the architecture to integrate with partnered APIs like heart trackers or open-source maps.

3 | Clarifications

The tech lead of the project contradicted herself in the first few sentences. She expresses her will to deliver a smooth and responsive experience. The benchmark for this would be 3 seconds, according to Odile, which is a long time and will not feel smooth at all. However, since she specifically mentioned 3 seconds, we will keep our application actions limited to those 3 seconds and strive for a quicker response.

The reason to use an actively maintained framework and programming language is mainly to make sure that future developers who want to work on the application will have enough experience to start directly, instead of learning a new and deprecated framework/language.

The class provided for communication with the heart monitors is assumed to be sufficient to communicate correctly with the API exposed by the monitors. No additional communication mechanism is assumed to be required.

All application data is assumed to be stored in the cloud, with the exact cloud storage solution to be specified later. Data may be stored temporarily on the device when the application is operating in offline mode. This temporary data will be synchronised with the cloud once a connection is available.

A "normal condition battery" is defined as a battery with at least 80% of its original capacity and an age of no more than 5 years from its manufacturing date.

Users can send friend requests to other registered users. A friendship is established only after the recipient accepts the request. Users can reject pending requests and remove existing friends.

The AI trainer will provide training recommendations, workout feedback and full training plans based on the data available to the application. The AI trainer is not intended to provide medical diagnoses or medical advice.

The documentation will include a description of how the external API's are used in order to keep the code of the application maintainable once the external API endpoints change.

4 | Domain model

Lidija Dudek

Based on the user story of Lidija (Section 1.1), the following domains are important:

- Starting/Ending Activities
- Tracking location
- Sharing activities
- Connecting with friends

The following model was created:

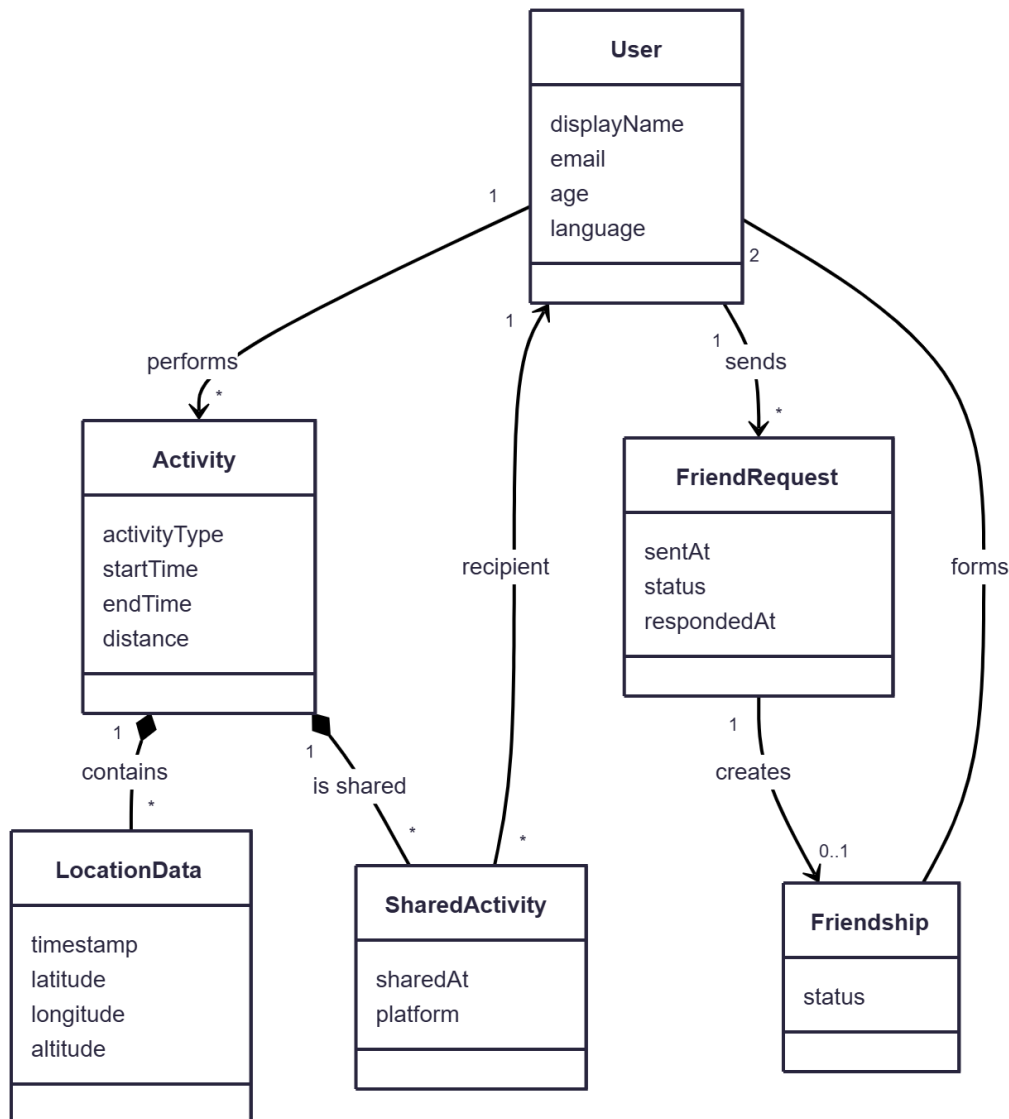


Figure 1: Domain model of Lidija (A.1)

Alexandria Kozlova

Based on the user story of Alexandria (Section 1.3), the following domains are important:

- Language settings
- Tracking location (accurately)
- Onboarding
- Connecting with other smart devices

The following model was created:

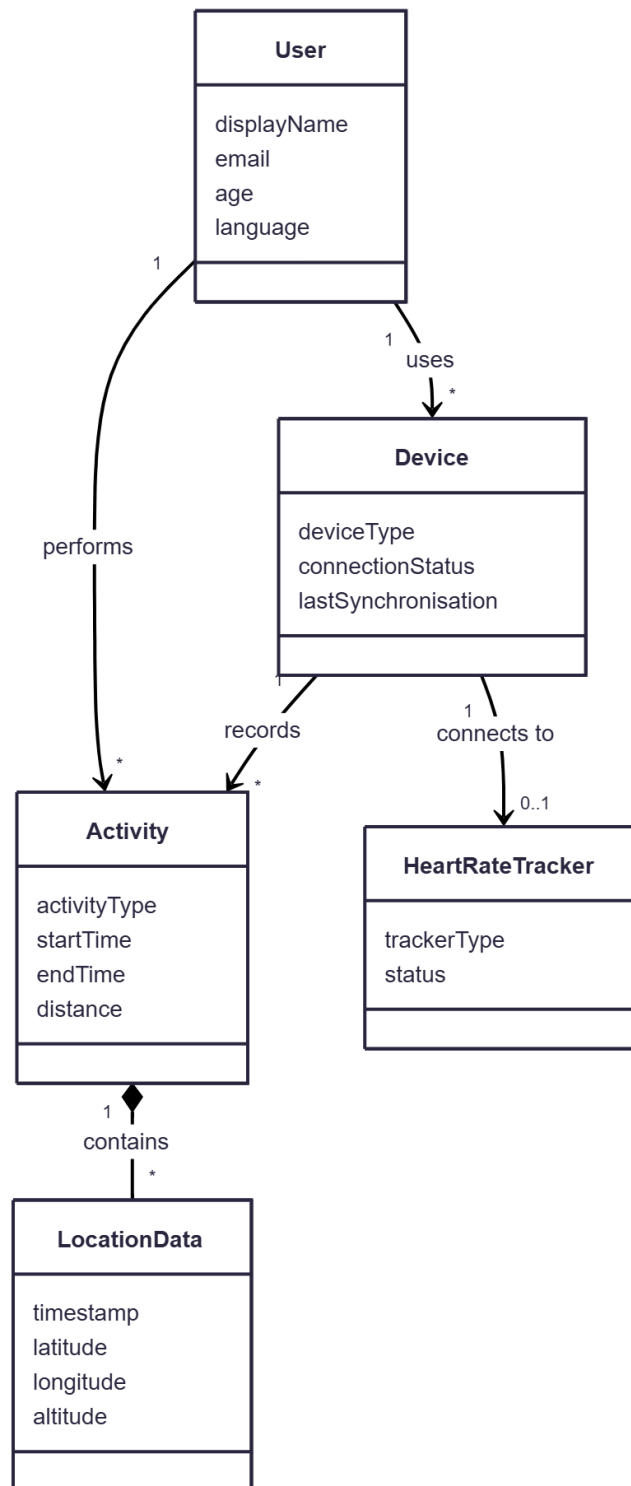


Figure 2: Domain model of Alexandria (A.2)

5 | Activity diagram

Track Activity

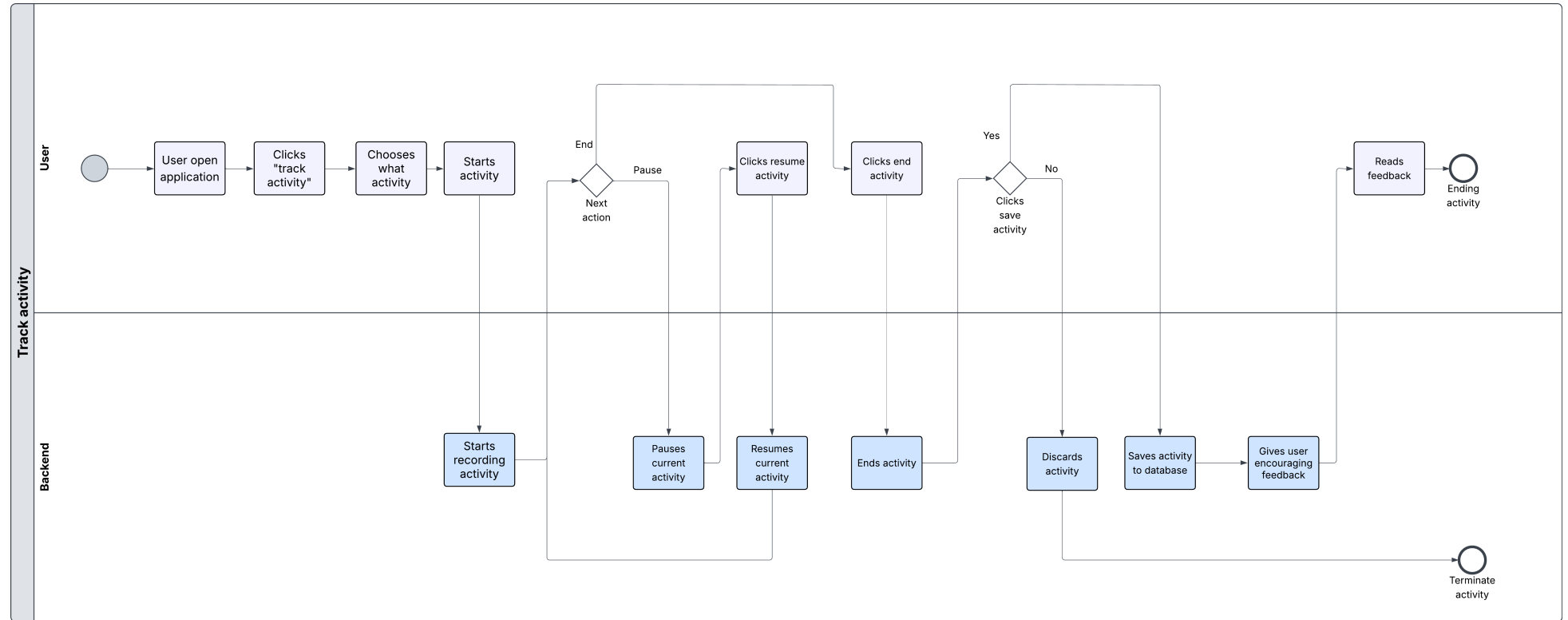


Figure 3: Activity diagram for tracking an activity

Friends and sharing

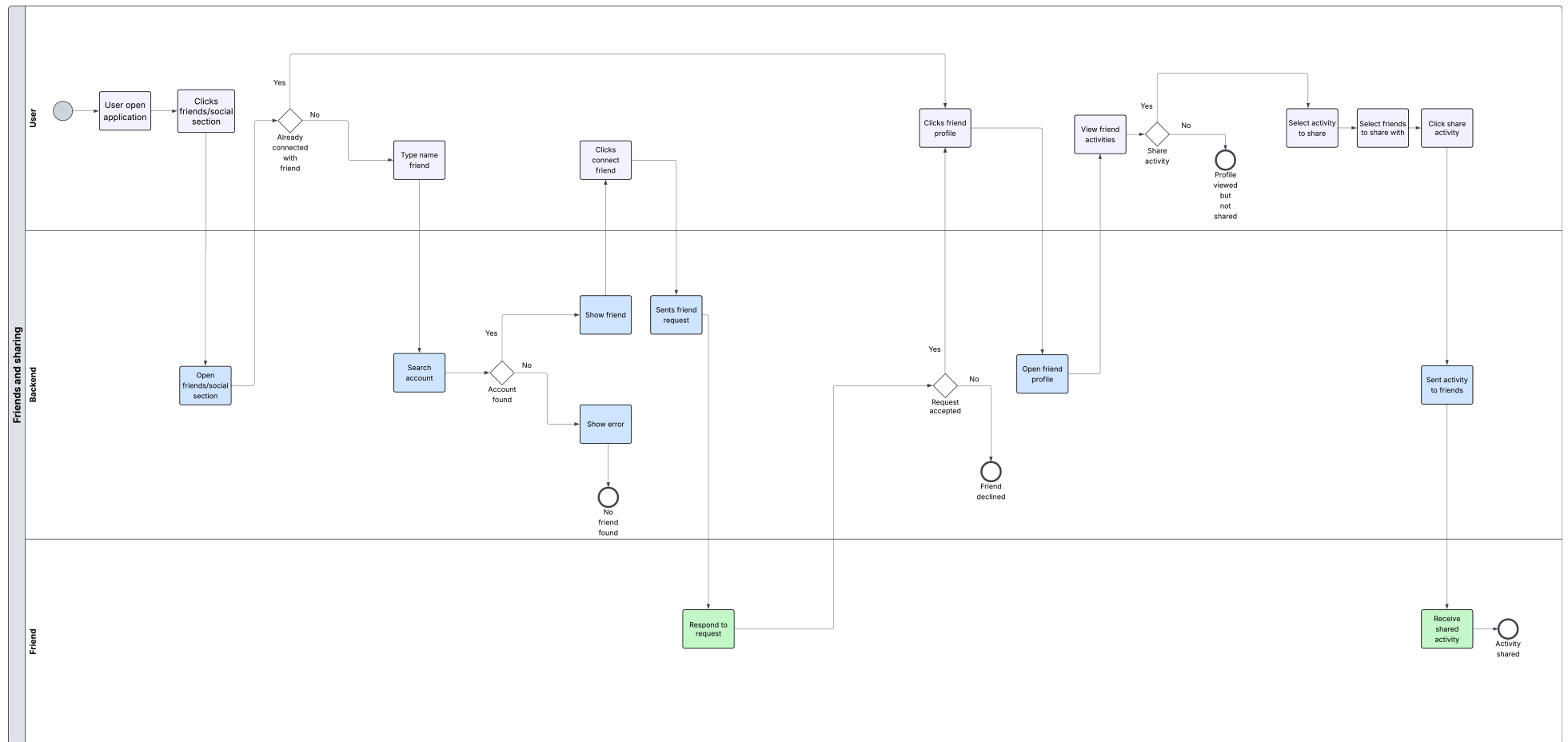


Figure 4: Activity diagram of friendships and activity sharing

6 | Software design

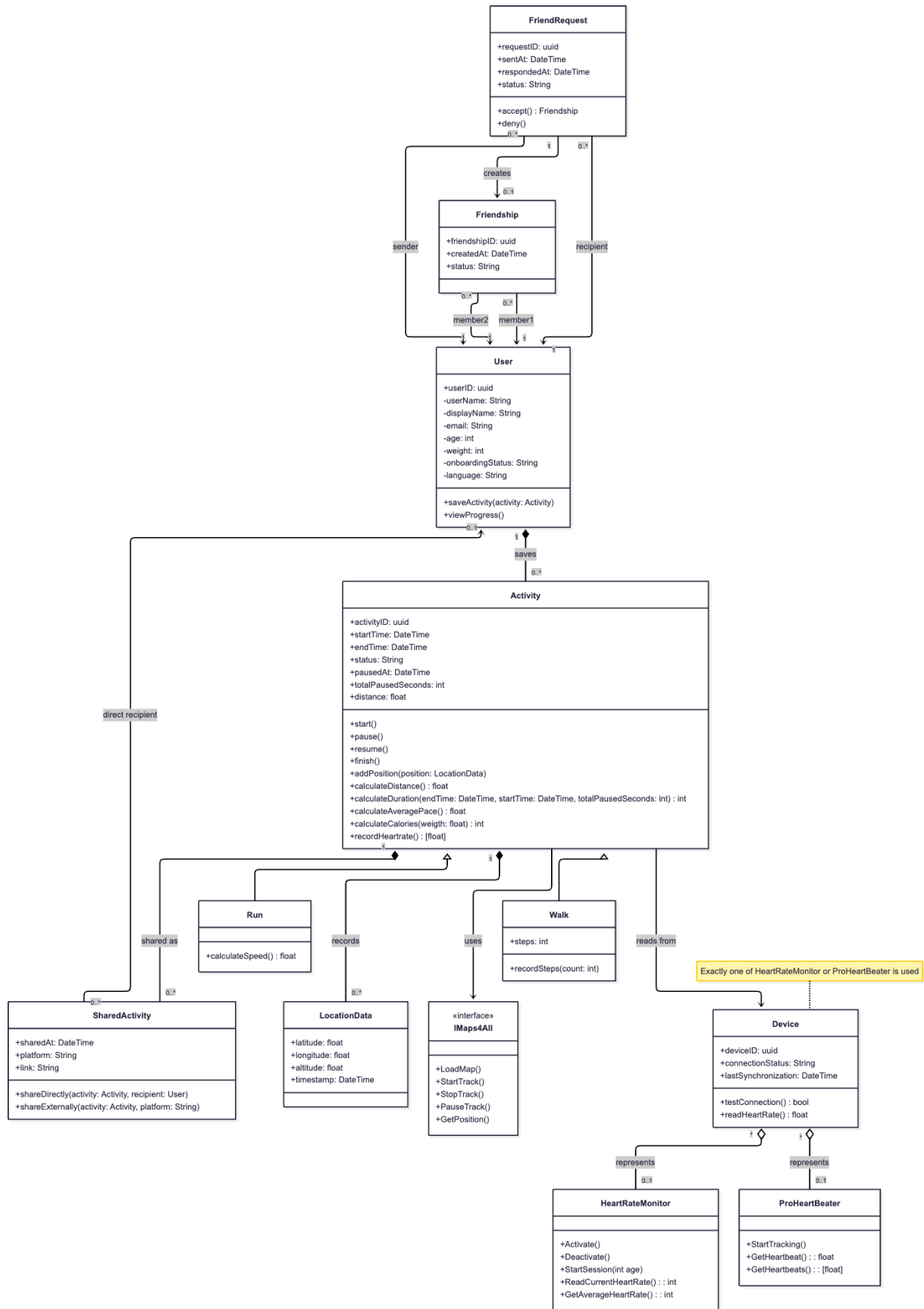


Figure 5: Class diagram A.3

In the class diagram, it is stated that a device is either a `HeartRateMonitor` or a `ProHeartBeater`. In reality, a device can be way more than one of these 2 hard trackers. However, in order to keep the project manageable, a device will be either of these for now. `readHeartRate()` will get data from either `ReadCurrentHeartRate()` or `GetHeartbeat()`, depending on what device is being used. In order to use the same `readHeartRate()` function for both APIs, we translate the integers outputted by `HeartRateMonitor` into floating-point numbers.

7 | Evaluation

Variation

Since it is a sport application, it should be possible to handle different types of activities. The clearest variation point is the type of activity. `Walk` and `Run` inherit from `Activity`, which means that additional activity types can be introduced as subclasses without changing the existing `Walk` and `Run` classes. Due to this design, it is possible to accommodate more types than currently implemented.

There are several additional variation points that are not explicitly represented in the current design (but are explained in future changes (Section 7.2)). Sharing platforms are one example. `SharedActivity` currently contains the sharing functionality, but different platforms could require different implementations. A common sharing interface could therefore be used so that platforms can be added without changing the core sharing logic.

Language is also a variation point because the application could support multiple languages. User-facing text should therefore not be hard-coded throughout the application. A language or localisation service could provide the required translations.

Future changes

The changes described in this section have not been implemented as part of the current design. They are proposed improvements that are intended to be implemented in future development iterations. The descriptions below therefore focus on the intended software design and how these changes could be integrated into the existing system.

Enhanced Activities

The first proposed future change is to expand the currently limited activity library. Currently, the `Activity` class contains position, location, and distance-related data. While this information is useful for activities such as walking and running, it is not relevant to every possible type of activity. Keeping this data in the base `Activity` class would therefore make it unnecessarily dependent on movement-based activities and could make it more difficult to introduce other types of activities in the future.

To make the activity system more flexible, the location- and movement-specific functionality should be separated from the general `Activity` class. This could be achieved through the following changes to the software design:

1. Position, location, and distance-related data should be removed from the `Activity` class.
2. A new subclass called `MovementActivity` should be introduced. This class would contain the position, location, and distance-related functionality removed from `Activity`.
3. The `Walk` and `Run` classes should inherit from `MovementActivity` instead of directly from `Activity`.

This separation would allow the `Activity` class to represent activities more generally, without assuming that every activity involves movement between locations. For example, activities that consist primarily of stationary movements could be implemented using a separate `StaticMovementActivity` subclass. Other activity types could similarly be introduced without having to include irrelevant location or distance-related functionality.

Activity-specific Statistics

The second future change should be separating activity data and lifecycle management from the calculation of activity-specific statistics. Currently, the `Activity` class is responsible for several calculations, such as duration, distance, average pace and calories.

To address this, an `IActivityStatistics<T>` interface should be introduced. The generic type `T` represents the activity for which the statistics are calculated. This allows each activity type to have its own statistics implementation without requiring the base `Activity` class to contain calculations for every possible activity.

For example, movement-based activities could use a **MovementStatistics** implementation to calculate values such as distance, pace and speed, while static activities could use a separate implementation containing statistics relevant to those activities. More specific activities can also provide their own implementation when necessary. For example, a **Yoga** activity could be associated with a **YogaStatistics** implementation without modifying the existing activity hierarchy. The software design should therefore be changed as follows:

1. A generic **IActivityStatistics<T>** interface should be introduced.
2. Calculations that are currently part of **Activity**, such as distance, pace and calories, should be moved to appropriate statistics implementations.
3. **MovementActivity** can use a **MovementStatistics** implementation for statistics that are shared between movement-based activities.

Notification center

The third and final future change would be the addition of a notification system. Instead of having **SharedActivity** handle the logic for each individual communication medium, this responsibility could be separated into a dedicated notification service and its implementations.

The following changes would be needed:

1. Create an **INotificationService** interface that defines the functionality required to send a notification.
2. Create implementations such as **InAppNotificationService**, **PushNotificationService**, and potentially **SMSNotificationService**.
3. Move the communication logic from **SharedActivity** into the appropriate notification service.

Each implementation would handle the logic required to deliver a notification through its specific medium. This would make it possible to add additional communication channels without modifying the existing **SharedActivity** class. As a result, the sharing functionality would be separated into smaller, more focused classes, making the system easier to extend and reducing the risk that changes to one communication method will affect the others.

Cohesion en coupling

Cohesion

When designing the class diagram, we already focused a great deal on decentralising both variables and functionalities to keep them close to their classes. This means that almost every class has every object it needs, and nothing more or less. However, when reviewing again, the conclusion is that the activity class is still quite big. Therefore, all calculation functions can be moved to different classes. For further explanation of this concept, look at both 7.2.1 and 7.2.2.

Coupling

First, every class is connected to at least one other class. This means that there are no *floating* classes, which is good. Almost every class depends on the **User** class, either directly or indirectly (through **Activity**), but in our case that is not a problem. Since this class diagram models an app that is behind a user login, it is evident that everything depends on a user. Exceptions to this are the **IMaps4All** and the **Device**. This can be explained since the same maps will be used for every user, and the same device can be shared between users.

8 | Work distribution & reflection

Work distribution

We divided the work for this project equally between the two group members. We both worked on different parts of the assignment, investing approximately the same amount of time and effort. The parts that were developed individually were always reviewed by the other person in order to minimise mistakes. Ultimately, we both contributed equally to the final result.

At the start of the assignment, we established clear agreements regarding deadlines to ensure the collaboration ran as smoothly as possible. The following schedule was used:

Planning

Part	Date finished	Group member
User stories	18-09-2026	Matt + Max
Non-function requirements	20-09-2026	Matt
Clarifications	20-09-2026	Max
Domain model	23-09-2026	Matt + Max
Activity diagram	23-09-2026	Max
Software design	25-09-2026	Max
Evaluation	27-09-2026	Matt
Work distribution	27-09-2026	Matt
Reflection	27-09-2026	Matt

Reflection

We had worked on several assignments together before, so we already had a good understanding about what we could expect from one another. Therefore, dividing the tasks went smoothly. There was a minor miscommunication regarding what program to use for the activity diagram, resulting in two different styles initially, but this was quickly resolved. Apart from that, we didn't encounter any real problems during the project.

We both put in a strong effort and had a clear understanding of what to expect from each other regarding quality, workload and collaboration. This made working together on this project much easier. For future projects, we could try to avoid these kinds of small miscommunications like we had with the style of the diagrams, but otherwise, we are satisfied with how the collaboration went.

A | Mermaid diagrams

Lidija Dudek

```
classDiagram
```

```
class User {
    displayName
    email
    age
    language
}
```

```
class Activity {
    activityType
    startTime
    endTime
    distance
}
```

```
class LocationData {
    timestamp
    latitude
    longitude
    altitude
}
```

```
class FriendRequest {
    sentAt
    status
    respondedAt
}
```

```
class Friendship {
    status
}
```

```
class SharedActivity {
    sharedAt
    platform
}
```

```
User "1" --> "*" Activity : performs
Activity "1" *-- "*" LocationData : contains
User "1" --> "*" FriendRequest : sends
FriendRequest "1" --> "0..1" Friendship : creates
User "2" -- Friendship : forms
Activity "1" *-- "*" SharedActivity : is shared
SharedActivity "*" --> "1" User : recipient
```

Alexandria Kozlova

classDiagram

```
class User {
    displayName
    email
    age
    language
}
```

```
class Activity {
    activityType
    startTime
    endTime
    distance
}
```

```
class LocationData {
    timestamp
    latitude
    longitude
    altitude
}
```

```
class Device {
    deviceType
    connectionStatus
    lastSynchronisation
}
```

```
class HeartRateTracker {
    trackerType
    status
}
```

```
User "1" --> "*" Activity : performs
Activity "1" *-- "*" LocationData : contains
User "1" --> "*" Device : uses
Device "1" --> "*" Activity : records
Device "1" --> "0..1" HeartRateTracker : connects to
```

Class diagram

classDiagram

```
class User {
    +userID: uuid
    -userName: String
    -displayName: String
    -email: String
    -age: int
    -weight: int
    -onboardingStatus: String
    -language: String
    +saveActivity(activity: Activity)
    +viewProgress()
}

class Activity {
    +activityID: uuid
    +startTime: DateTime
    +endTime: DateTime
    +status: String
    +pausedAt: DateTime
    +totalPausedSeconds: int
    +distance: float
    +start()
    +pause()
    +resume()
    +finish()
    +addPosition(position: LocationData)
    +calculateDistance() float
    +calculateDuration(endTime: DateTime, startTime: DateTime, totalPausedSeconds: int) int
    +calculateAveragePace() float
    +calculateCalories(weighth: float) int
    +recordHeartrate () [float]
}

class Run {

    +calculateSpeed() float
}

class Walk {
    +steps: int
    +recordSteps(count: int)
}

class LocationData {
    +latitude: float
    +longitude: float
    +altitude: float
    +timestamp: DateTime
}

class IMaps4All {
    <<interface>>
    +LoadMap()
    +StartTrack()
    +StopTrack()
}
```

```

    +PauseTrack()
    +GetPosition()
}

class Device {
    +deviceId: uuid
    +connectionStatus: String
    +lastSynchronization: DateTime
    +testConnection() bool
    +readHeartRate() float
}

class HeartRateMonitor{
    +Activate()
    +Deactivate()
    +StartSession(int age)
    +ReadCurrentHeartRate() : int
    +GetAverageHeartRate() : int
}

class ProHeartBeater {
    +StartTracking()
    +GetHeartbeat() : float
    +GetHeartbeats() : [float]
}

class FriendRequest {
    +requestID: uuid
    +sentAt: DateTime
    +respondedAt: DateTime
    +status: String
    +accept() Friendship
    +deny()
}

class Friendship {
    +friendshipID: uuid
    +createdAt: DateTime
    +status: String
}

class SharedActivity {
    +sharedAt: DateTime
    +platform: String
    +link: String
    +shareDirectly(activity: Activity, recipient: User)
    +shareExternally(activity: Activity, platform: String)
}

User "1" *-- "0..*" Activity : saves
Activity "1" *-- "0..*" LocationData : records
Activity "1" *-- "0..*" SharedActivity : shared as
Activity <|-- Run
Activity <|-- Walk
Activity --> IMaps4All : uses
Activity --> Device : reads from
Device "*" o-- "0..1" HeartRateMonitor : represents
Device "*" o-- "0..1" ProHeartBeater : represents

```

```
FriendRequest "0..*" --> "1" User : sender
FriendRequest "0..*" --> "1" User : recipient
FriendRequest "1" --> "0..1" Friendship : creates
Friendship "0..*" --> "1" User : member1
Friendship "0..*" --> "1" User : member2
SharedActivity "0..*" --> "0..1" User : direct recipient
```

note for Device "Exactly one of HeartRateMonitor or ProHeartBeater is used"